Università degli Studi di Milano
FACOLTÀ DI SCIENZE E TECNOLOGIE

DIPARTIMENTO DI INFORMATICA
GIOVANNI DEGLI ANTONI

CORSO DI LAUREA TRIENNALE IN
INFORMATICA

IMPLEMENTAZIONE DIGITALE DI UN GIOCO DA TAVOLO CON GRAFICA INTERATTIVA PER IL POTENZIAMENTO DELLE COMPETENZE SOCIALI DI ADOLESCENTI AUTISTICI

Relatore: Prof. Alberto Borghese

Elaborato Finale di:
Samuel De Benedictis
Matr. Nr. 855264

Anno Accademico 2025-2026

Indice

Indice **i**

1 Introduzione **1**
1.1 L'autismo 1
1.2 Il template 2
1.3 I contenuti 2
1.4 Organizzazione della tesi 3
1.5 Stile e forma 4

2 Stato dell'arte **6**
2.1 Risorse 6
2.2 Buone pratiche 7
2.3 Bibliografia e sitografia 8

3 Tecnologie utilizzate **9**
3.1 Next.js 15 9
3.2 TypeScript 9
3.3 Zustand 10
3.4 TailwindCSS 10
3.5 Biome 10
3.6 Vitest 11
3.7 Playwright 11
3.8 Storybook 11
3.9 Husky 11
3.10 Commitlint 12
3.11 Considerazioni sull'architettura tecnologica 12

4 Nome del Progetto **13**
4.1 Panoramica del progetto 13
4.2 Implementazione 13

5 Test **14**
5.1 Protocollo 14
5.2 Risultati 14
5.3 Osservazioni 15

6 Conclusioni **16**
6.1 Conclusioni 16
6.2 Sviluppi futuri 16

A Informazioni generali sull'attività di tirocinio **17**
A.1 Aspetti Burocratici (CdL Triennali) 18
A.2 Aspetti Pratici 18
A.3 Gestione del Codice 19

B Documenti da Produrre **20**
B.1 Riassunto 20
B.2 Presentazione 21

Bibliografia **23**

Capitolo 1

Introduzione

1.1 L'autismo

"Autistico" è un aggettivo coniato dallo psichiatra svizzero Eugen Bleur per descrivere la chiusura in sé stessi di alcuni suoi pazienti (dal greco *autos*) ed in seguito viene utilizzato da molti autori per descrivere difficoltà osservate in pazienti bambini e adolescenti.

La psichiatra Grunya Efimovna Sukhareva nel 1926 descrive dettagliatamente le caratteristiche autistiche di sei bambini, che tuttavia vengono diagnosticati come affetti da psicopatia schizoide.

Solo nel 1943 viene proposta una distinzione dell'autismo da altre sindromi. Il pediatra Hans Asperger, nella sua tesi di dottorato, utilizza il termine "psicopatia autistica" e descrive accuratamente la condizione di alcuni ragazzi basandosi sull'osservazione di circa duecento pazienti. Leo Kanner nello stesso anno riporta le osservazioni compiute su undici bambini caratterizzate da "marcate difficoltà nelle relazioni sociali e nella comunicazione", ed in un lavoro successivo propone di chiamare questa sindrome "autismo infantile".

Negli anni Settanta, Lorna Wing fornisce le prime prove riguardo alla natura composita dell'autismo e l'impossibilità di descriverlo in base a criteri diagnostici univoci ed introduce l'espressione "disturbo dello spettro autistico".

Come sottolineato da Luca Surian, il termine "spettro" mette in risalto la concezione dimensionale che era stata proposta da Asperger, questa prospettiva sottolinea la variabilità quantitative e graduale, piuttosto che la differenza qualitativa e categoriale; secondo questa prospettiva, la differenza tra persone con o senza autismo va pensata come una differenza di grado, non di qualità. [1] L'autismo viene diagnosticato quando una costellazione di tratti è presente nello stesso individuo e si manifesta in forma molto accentuata.

Alcuni di questi tratti sono: propensione per la ripetizione di alcune attività, spiccata attenzione ai dettagli e difficoltà nell'accettare cambiamenti.

1.2 Il template

Se questo documento è stato ottenuto tramite condivisione diretta di file, è consigliabile accedere alla versione continuamente aggiornata e disponibile su Overleaf.

`https://www.overleaf.com/read/hmffzxzhhdqn`

Questo template è stato sviluppato, negli anni, dai membri del Laboratorio di Informatica Musicale dell'Università degli Studi di Milano, in particolare da: Giorgio Presti, Luca Andrea Ludovico, Federico Avanzini, e Marco Tiraboschi.

È stato principalmente inteso per gli elaborati finali del corso di laurea triennale in Informatica Musicale, e poi esteso anche agli altri CdL del Dipartimento di Informatica, ma può essere riadattato anche per altri corsi cambiando i metadati nel preambolo. Nel resto del documento, dove non specificato, useremo il termine *tesi* nella sua accezione generica che include anche gli elaborati triennali.

1.3 I contenuti

Alcune volte, le tesi possono avere forti connotazioni interdisciplinari. È però fondamentale ricordarsi che si tratta di lavori in area informatica, e questo aspetto deve emergere con chiarezza. Anche gli elaborati di contenuto più umanistico devono mostrare rigore scientifico e uno sforzo di formalizzazione nel loro svolgimento. In concreto, sono molto apprezzati schemi, tabelle, formalismi grafici e/o matematici, presenza di parti significative di codice (ove possibile).

Uno scritto di questa estensione parte sempre da un punto pregresso, tipicamente l'analisi dello stato dell'arte e della letteratura. È poi richiesto un contributo personale (e originale, nel caso delle tesi magistrali) per giungere a una conclusione formulata dall'autore, eventualmente anche in contrasto con il pensiero corrente o con quanto ci si prefigge all'inizio dell'opera.

Le tre fasi devono emergere con chiarezza:

- analisi dello stato dell'arte e/o della letteratura;
- contributo personale alla ricerca;
- conclusioni raggiunte, ed eventuali sviluppi futuri.

Nel loro complesso queste tre componenti devono rendere evidente la *rilevanza* del lavoro, in termini di

- chiarezza nella definizione del problema, delle motivazioni e degli obiettivi della tesi;

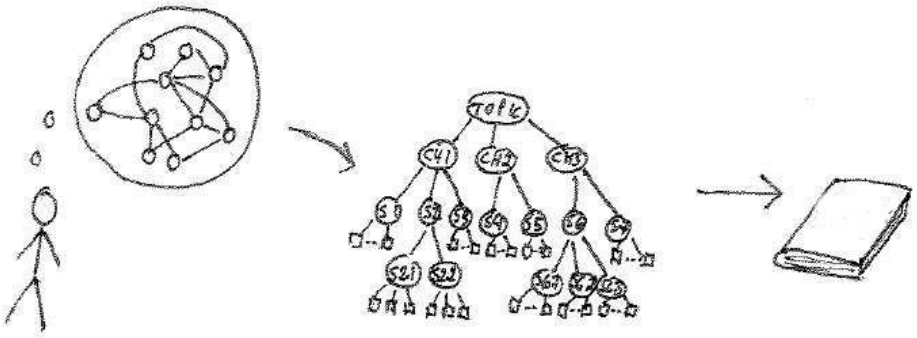

Figura 1: Il processo di trasformazione delle idee in testo [?].

- connessione con la letteratura (così da mostrare che la tematica oggetto della tesi è di interesse per la comunità scientifica);
- rilevanza della tematica nell'ambito delle discipline informatiche.

1.4 Organizzazione della tesi

La scelta di come strutturare un lavoro esteso, quale un elaborato finale o una tesi, non è semplice nè univoca, pertanto quelli sotto elencati vanno presi come suggerimenti generici e attualizzati alla propria situazione personale.

Prima di cominciare a scrivere testo è fondamentale creare un indice dei contenuti organizzato in maniera gerarchica (tesi, capitoli, sezioni, sottosezioni, ecc.). Una volta ottenuto un indice soddisfacente, questo potrà essere riempito dal testo della tesi. Un approccio informatico alla questione è il seguente: scrivere una tesi significa partire da un grafo di idee [?] e arrivare a creare un albero di contenuti testuali (un *minimum spanning tree* del grafo delle idee!). L'albero rappresenta la struttura del testo, dove il nodo radice è la tesi stessa, i suoi figli sono i capitoli, ecc. Le foglie dell'albero rappresentano le sottosezioni di livello più fine. Idealmente l'albero dei contenuti dovrebbe avere le seguenti caratteristiche:

- essere bilanciato;
- avere un'altezza $h = 4$ o 5;
- essere un albero n-ario con $2 \leq n \leq 7$

Anche i contenuti testuali di tutti i nodi allo stesso livello dell'albero dovrebbero essere il più possibile bilanciati (ad esempio, sarebbe sbagliato avere un capitolo di una pagina e

un altro di 30 pagine). È tollerato uno sbilanciamento a favore del capitolo che descrive il proprio lavoro specifico e i contributi innovativi.

Limitatamente al secondo livello dell'albero (capitoli), un esempio puramente indicativo di scaletta per una tesi di natura sperimentale è quello che segue:

- Indice
- Introduzione
- Capitolo sullo stato dell'arte
- Capitolo sulle tecnologie utilizzate[1]
- Capitolo sul caso di studio o sul software realizzato
- Capitolo sui test effettuati
- Breve capitolo su conclusioni e sviluppi futuri
- Bibliografia ed eventuale Sitografia
- Eventuali appendici (ad esempio, listati completi di codice, manuale utente, dimostrazioni, ecc.)

Sarà dunque opportuno prevedere, nel capitolo introduttivo, un esplicito richiamo alla struttura del documento. Ad esempio: "Il presente lavoro è organizzato come segue: nel Capitolo 1 ...".

1.5 Stile e forma

Lo stile di una tesi scientifica deve essere esatto, chiaro, compatto, oggettivo [**?**].

Esatto vuol dire che ogni parola utilizzata significa solo ciò che deve esprimere. È opportuno cercare di evitare sinonimi per riferirsi a concetti importanti. Ove possibile si definiscano variabili (la frequenza f, il tempo n, ecc.) e le si usino in maniera sistematica. Vanno evitate il più possibile espressioni vaghe e non quantitative ("abbastanza grande", "praticamente tutti", "molto pochi", ...). Si usino i pronomi con parsimonia, meglio ripetere qualche sostantivo nel testo piuttosto che lasciare delle ambiguità.

Chiaro significa che la struttura e la forma devono essere funzionali a trasmettere l'informazione nel modo più immediato e comprensibile. I titoli di capitoli e sezioni devono essere illustrativi dei contenuti (è anche utile avere un breve testo all'inizio di un capitolo o una sezione, prima di passare alle sottosezioni). Il testo deve essere diviso logicamente in

[1] Questo capitolo si può omettere se le sole tecnologie utilizzate sono strumenti standard come Python, JavaScript, C++, Matlab, ecc.

frasi, paragrafi, capoversi: sono preferibili paragrafi brevi, diretti e dichiarativi, limitando le subordinate e gli incisi, e i capoversi devono essere coerenti con la logica del discorso. L'uso di termini tecnici va limitato solo ai casi necessari. Anche gli acronimi devono essere usati con parsimonia, e vanno spiegati la prima volta che vengono usati.

Compatto significa che si deve scrivere solo ciò che è necessario scrivere. Bisogna evitare ripetizioni di concetti e osservazioni. Bisogna limitare descrizioni e dettagli che non siano necessari alla comprensione del discorso generale. In particolare vanno evitate osservazioni banali, che risultino ovvie al lettore. Tra una forma verbosa ("in considerazione del fatto che…") e una equivalente più sintetica ("perché…") si preferisca la seconda.

Oggettivo significa che ciò che si scrive deve essere privo di elementi soggettivi e di elementi che possono influenzare la valutazione. Vanno accuratamente evitate considerazioni personali ("Questa idea mi è venuta per la prima volta in occasione…"), nonché affermazioni opinabili il cui giudizio sia lasciato al parere personale dell'autore. L'unico modo per supportare tali prese di posizione è fornire dati scientifici a riprova della propria tesi, o riferirsi in modo esplicito a un antecedente bibliografico ("ipse dixit") citandone la provenienza in bibliografia. Infine, l'elaborato deve essere steso in forma impersonale. Frasi quali "Durante la mia esperienza ho approfondito i temi…" sono facilmente sostituibili con locuzioni quali "Durante la fase di analisi sono stati approfonditi i temi…".

Un uso appropriato di figure e tabelle è essenziale per supportare i punti appena discussi. Figure e tabelle devono avere delle didascalie autoesplicative (che non richiedano di leggere il testo principale per capirne il significato). Il testo principale deve però sempre contenere un riferimento alla figura o alla tabella ("Come mostrato in Fig.1, …"). L'uso di schemi grafici (uno schema a blocchi di algoritmo di elaborazione audio, uno schema concettuale di progetto software, ecc.) è particolarmente utile a supportare la chiarezza e la compattezza dell'esposizione.

Capitolo 2

Stato dell'arte

Una delle domande più ricorrenti da parte degli studenti è cosa si intenda per "stato dell'arte". Concretamente, si tratta di individuare la situazione corrente riguardo la tematica trattata nell'elaborato. A titolo di esempio, se il lavoro di tesi si concentra su un (presunto) innovativo algoritmo per suggerire i brani di una playlist, è necessario verificare che tale approccio sia veramente innovativo e, in ogni caso, studiare gli approcci alternativi già disponibili in letteratura.

Per ottenere tale risultato, è necessario condurre ricerche approfondite a livello bibliografico (testi di riferimento, articoli scientifici, ecc.) e implementativo (sul web, sugli store, ecc.) **prima** di iniziare il proprio lavoro.

2.1 Risorse

Riguardo gli articoli scientifici, si caldeggia l'uso del motore di ricerca specializzato Google Scholar.[1] Esistono, poi, numerosi repository che consentono di accedere gratuitamente a pubblicazioni scientifiche, tra cui ResearchGate,[2] Zenodo,[3] e SBA - Sistema Bibliotecario di Ateneo.[4] In particolare quest'ultimo permette agli studenti dell'Ateneo di accedere gratuitamente a molte pubblicazioni altrimenti accessibili solo a pagamento.

È importante riconoscere, nella grande mole di letteratura scientifica disponibile, i lavori più autorevoli e affidabili. Un primo elemento per orientarsi è che non tutte le sedi di pubblicazione sono uguali. In particolora una regola generale (per quanto non assoluta) è che gli articoli pubblicati su *rivista* sono tipicamente più completi e rigorosi di quelli pubblicati su atti di *convegno*. Un secondo elemento correlato all'autorevolezza della pubblicazione è il numero di citazioni che questa ha raccolto da parte di altri ricercatori (dato

[1] `https://scholar.google.it/`.

[2] `https://www.researchgate.net/`

[3] `https://zenodo.org/`

[4] `http://www.sba.unimi.it/index.html` con contenuti scaricabili solo dalla rete interna all'ateneo.

visibile ad esempio su Google Scholar). Infine, è opportuno riconoscere le sedi di pubblicazione (riviste e convegni) rilevanti per il settore: per questo, si consiglia di confrontarsi con i propri relatori e di consultare indicatori di qualità come Scimago JR[5] per le riviste e il GII-GRIN-SCIE Conference Rating[6] per le conferenze.

2.2 Buone pratiche

Alcuni consigli di buone pratiche per l'analisi dello stato dell'arte.

- Imparare a leggere in maniera *efficace*. Ci sono migliaia di articoli potenzialmente interessanti, quindi è essenziale riuscire a estrarre in breve tempo gli elementi rilevanti di un articolo senza perdersi nei dettagli (fino a che non sia strettamente necessario), nonché annotare in maniera ordinata tali elementi.
- Imparare a leggere in maniera *critica*. Raramente un articolo scientifico è perfetto, gli aspetti tecnici e gli eventuali punti deboli (metodologie, ripetibilità dei risultati, ecc.) vanno valutati e annotati con attenzione.
- Imparare a seguire le "piste" interessanti. Una volta trovato un articolo rilevante, è utile esaminare
  - gli articoli che esso cita (i suoi riferimenti bibliografici): questo permette di rintracciare riferimenti autorevoli a cui l'articolo si appoggia;
  - gli articoli che lo citano (ad esempio Google Scholar offre questa funzionalità): questo permette di rintracciare riferimenti più recenti che hanno proseguito nella stessa direzione di ricerca.
- Imparare a lavorare iterativamente.
  - Aggiungere alla propria bibliografia gli articoli considerati rilevanti, a mano a mano che vengono trovati.
  - Raggrupparli iterativamente in sottotematiche.
  - Usare inizialmente un approccio "inclusivo" (nel dubbio, aggiungere un articolo in bibliografia piuttosto che scartarlo), e solo in un secondo tempo decidere cosa tenere e cosa scartare.

[5] `https://www.scimagojr.com`
[6] `https://scie.lcc.uma.es:8443`

2.3 Bibliografia e sitografia

La bibliografia di un lavoro scientifico deve necessariamente essere ricca. Tutti i testi in bibliografia devono essere citati almeno una volta nel corso dell'elaborato.

Nella valutazione della bibliografia da parte della commissione, i testi cartacei sono considerati significativamente più validi dei siti consultati. Ne consegue che la bibliografia debba essere ricca di testi pubblicati, siano essi libri, articoli, al limite tesi di laurea o di dottorato o rapporti tecnici. Si suggerisce di evitare citazioni a fonti di dubbia valenza scientifica, quali Wikipedia, W3Schools, ecc.

Se è necessario citare siti Web, esistono tre strade ugualmente accettabili:

1. se il numero di siti non è preponderante rispetto ai testi "tradizionali", è possibile inserirli parimenti in bibliografia;

2. se i siti da citare sono numerosi, è più opportuno creare una sorta di bibliografia parallela e separata, detta *sitografia*;

3. infine, se la citazione dei siti serve a individuare un prodotto e non una fonte di informazioni, la soluzione più opportuna è quella delle note a piè di pagina.

Capitolo 3

Tecnologie utilizzate

Questo capitolo descrive gli strumenti tecnologici utilizzati nello sviluppo del progetto. L'applicazione sviluppata è scritta in linguaggio TypeScript col framework Next.js, a sua volta basato su React; una volta compilato e buildato restituisce una web app interpretabile dalla maggioranza dei browser sul mercato. L'output del processo è un set di pagine web (con relativi file JavaScript e CSS) che possono essere navigate da browser e forniscono la view, i controlli e il modello del gioco da tavolo creato.

La maggior parte di questi framework e librerie è stata scelta per garantire qualità del codice, esperienza di sviluppo efficiente e manutenibilità a lungo termine.

3.1 Next.js 15

Next.js è un framework React per applicazioni full-stack che offre rendering lato server (SSR), generazione di siti statici (SSG) e routing automatico. Fornisce un'infrastruttura completa per applicazioni web moderne, gestendo aspetti come routing, ottimizzazione delle immagini, gestione degli asset e deployment. La versione 15 introduce ulteriori ottimizzazioni delle performance e una migliore integrazione con gli strumenti di sviluppo.

È stato scelto come framework principale per la sua capacità di gestire in modo nativo sia applicazioni statiche che dinamiche, offrendo un'ottima esperienza di sviluppo e performance elevate per l'utente finale.

3.2 TypeScript

TypeScript è un superset di JavaScript che aggiunge tipizzazione statica al linguaggio. Questo strumento permette di identificare errori a tempo di compilazione, migliorando significativamente la qualità e la sicurezza del codice. La tipizzazione aiuta anche nel

refactoring e nell'autocompletamento, rendendo lo sviluppo più rapido e meno soggetto a bug.

La sua integrazione con Next.js è perfetta, permettendo di sfruttare tutti i vantaggi della tipizzazione statica in un progetto React complesso come un gioco da tavolo digitale.

3.3 Zustand

Zustand è una libreria leggera per la gestione dello stato dell'applicazione che offre un'alternativa semplice e performante a soluzioni più complesse come Redux. Si basa su un approccio minimalista che permette di creare store di stato in modo intuitivo, senza bisogno di boilerplate e con un'ottima integrazione TypeScript.

È stato adottato per gestire lo stato globale del gioco, come la posizione dei giocatori, lo stato del turno e le informazioni sui mazzi di carte, semplificando notevolmente la gestione dello stato rispetto ad altre soluzioni più pesanti.

3.4 TailwindCSS

TailwindCSS è un framework CSS utility-first che fornisce classi CSS modulari e riutilizzabili. Invece di scrivere CSS personalizzato, si utilizzano classi predefinite direttamente nel markup HTML/JSX.

Questa scelta ha permesso di velocizzare notevolmente lo sviluppo dell'interfaccia utente, garantendo coerenza visiva e semplificando la creazione di layout responsive senza dover scrivere codice CSS complesso.

3.5 Biome

Biome è uno strumento moderno che combina in un unico pacchetto le funzionalità di linting, formattazione del codice e type checking. Progettato per essere estremamente veloce e accurato, sostituisce tool tradizionali come ESLint e Prettier con un'unica configurazione. Offre un'esperienza di sviluppo più fluida grazie all'integrazione perfetta tra le diverse funzionalità.

La sua adozione ha migliorato significativamente l'esperienza di sviluppo con linting e formattazione istantanei, riducendo il tempo dedicato alla configurazione e manutenzione degli strumenti di qualità del codice.

3.6 Vitest

Vitest è un framework di testing moderno e veloce, progettato per essere utilizzato con Vite. Offre un'esperienza di testing simile a Jest ma con prestazioni superiori grazie all'integrazione nativa con Vite. Supporta il testing unitario e l'analisi della copertura del codice.

Viene utilizzato per verificare il corretto funzionamento delle singole unità di codice in modo rapido ed efficiente, garantendo la qualità del software attraverso test automatizzati.

3.7 Playwright

Playwright è un framework per testing end-to-end che supporta multi-browser. Permette di automatizzare browser reali per testare il comportamento completo dell'applicazione dall'interfaccia utente fino alla logica di backend.

È impiegato per simulare il comportamento reale degli utenti, verificando che tutte le funzionalità dell'applicazione funzionino correttamente insieme e su diversi browser.

3.8 Storybook

Storybook è uno strumento per lo sviluppo e la documentazione di componenti UI in isolamento. Permette di creare una libreria visiva dei componenti dell'applicazione, mostrandoli in diversi stati e configurazioni.

Viene utilizzato per facilitare lo sviluppo dei componenti dell'interfaccia in modo indipendente, fornire documentazione visiva e permettere di verificare il corretto rendering in diversi contesti.

3.9 Husky

Husky è una libreria per la gestione dei Git hooks che permette di automatizzare operazioni come linting, testing e validazione del codice prima dei commit. Configurando hooks specifici, è possibile garantire che ogni modifica al codice rispetti gli standard di qualità stabiliti dal team di sviluppo.

La sua integrazione con il flusso di lavoro Git automatizza i controlli di qualità del codice, prevenendo l'introduzione di errori e garantendo coerenza nello stile di codifica.

3.10 Commitlint

Commitlint è uno strumento per validare i messaggi di commit secondo convenzioni stabilite. Permette di definire regole per il formato, la lunghezza e il contenuto dei messaggi di commit, migliorando la qualità e la consistenza della cronologia del progetto. Si integra perfettamente con Husky per validare i commit in tempo reale.

La sua adozione migliora la qualità della cronologia del progetto e facilita la generazione automatica di changelog, rendendo più semplice il tracciamento delle modifiche e la gestione delle versioni.

3.11 Considerazioni sull'architettura tecnologica

L'architettura tecnologica scelta combina strumenti moderni e ben collaudati che insieme garantiscono:

- **Qualità del codice** attraverso tipizzazione, linting e testing approfondito
- **Esperienza di sviluppo** efficiente con strumenti performanti e integrati
- **Manutenibilità** grazie a codice ben strutturato e documentato
- **Affidabilità** attraverso test multi-livello che coprono sia singole unità che flussi completi

Questa combinazione permette di sviluppare un'applicazione complessa come un gioco da tavolo digitale mantenendo alti standard di qualità e facilitando la collaborazione tra sviluppatori.

Capitolo 4

Nome del Progetto

In genere il capitolo più corposo dell'elaborato è quello in cui si parla del lavoro svolto. Esistono alcune buone pratiche per rendere l'esposizione efficace, eccone alcune.

4.1 Panoramica del progetto

Prima di addentrarsi nei dettagli è bene fornire una panoramica (anche molto schematica, corredata da un diagramma) del lavoro svolto, in modo che il lettore abbia una mappa concettuale con cui orientarsi.

4.2 Implementazione

Una volta fornita una panoramica, è possibile addentrarsi nei dettagli, ricordando che si sta scrivendo un articolo scientifico e non un testo di narrativa. Vanno dunque evitate domande retoriche quali "*ma come è stato possibile risolvere questo problema? Ebbene …*".

Vale la pena riportare nel testo solo le parti di codice più importanti, demandando all'appendice il codice completo e altri extra come il manuale utente.

Capitolo 5

Test

Ogni lavoro scientifico richiede una validazione dei risultati ottenuti. Questo si può fare confrontando in modo sistematico il proprio lavoro con lavori concorrenti o misurando l'efficacia del lavoro mediante test con gli utenti. È fondamentale che questi test siano ripetibili, dovrete dunque fornire tutti i dettagli necessari nel testo per permettere a chi legge la tesi di replicare l'esperimento.

Progettare e condurre un test soggettivo con utenti è un lavoro complesso e lungo, che richiede pianificazione e competenza. Il bel libro di Lazar *et al.* [?] illustra in dettaglio i concetti principali della ricerca sperimentale e le metodologie correlate: ipotesi di ricerca, design sperimentale, analisi dei risultati sperimentali. Quelli che seguono sono alcuni consigli specifici sugli aspetti più importanti.

5.1 Protocollo

Uno degli errori più comuni è sottovalutare lo studio di un *protocollo* sperimentale. Affinché i risultati dei test siano significativi è necessario non trascurare i seguenti aspetti:

- eliminare distorsioni sistematiche involontarie;
- isolare le variabili oggetto dello studio;
- garantire una numerosità sufficiente del campione;
- confrontare gli effetti con un gruppo di controllo.

5.2 Risultati

I risultati dei test vanno presentati in modo chiaro e completo, possibilmente indicando la significatività statistica di quanto ottenuto.

Colonna 1	Colonna 2	Colonna 3
5	8	1
6	9	2
7	0	3

Tabella 1: Tabella di esempio.

È buona norma fornire sia i dati numerici (un esempio di come si fanno le tabelle in LaTeX è visibile in Tab. 1), sia una rappresentazione grafica (a barre, a scatole e baffi, a violino, di dispersione, ecc.).

È inoltre consigliato riportare in appendice i dati grezzi completi, in modo da permettere al lettore di ripetere eventuali test statistici.

5.3 Osservazioni

Quando si traggono conclusioni dai dati bisogna prestare attenzione a non confondere la correlazione con un rapporto di causalità. Molto spesso accade che un test suggerisca la presenza di un fenomeno, ma non dica nulla sulla causa. In questo caso bisogna formulare delle ipotesi, calcolare le implicazioni, ed eseguire un test che valuti se e quali di queste implicazioni si verifichino. Se il nuovo test falsifica la teoria, non importa quanto questa sia elegante: è falsa. Se invece il nuovo test non falsifica la teoria, allora la si può dare per "vera fino a prova contraria".

Per queste ragioni è necessario esporre le proprie osservazioni in maniera cauta, senza andare oltre ciò che suggeriscono i dati. È certamente possibile speculare sulle cause, ma va esplicitato chiaramente, e tali speculazioni vanno supportate dalla bibliografia.

Capitolo 6

Conclusioni

6.1 Conclusioni

Nelle conclusioni si tirano le somme di quanto realizzato, facendo un riassunto stringato del lavoro svolto. In particolare vanno dichiarati punti di forza e criticità della ricerca effettuata, nonché quali aspetti dello stato dell'arte siano stati superati dal lavoro in oggetto.

6.2 Sviluppi futuri

Tra gli sviluppi futuri in genere trovano posto quelle migliorie non realizzate per mancanza di tempo, la cui necessità è emersa solo dopo i test, e che riguardano il progetto ad un livello di astrazione più alto (nel caso di tesi che si inquadrano in una linea di ricerca).

Appendice A

Informazioni generali sull'attività di tirocinio

Lo studente che si appresta a svolgere il lavoro di tesi generalmente incontra relatore e correlatore a inizio progetto per definire una tabella di marcia, quindi al completamento di ogni obiettivo fissato si ripresenta per un aggiornamento e un controllo. Questo è molto importante per approcciare il problema in modo organico e per evitare di illudersi di aver raggiunto un risultato sufficiente ritenendo concluso il lavoro.

Durante il lavoro è consigliato di prendere appunti su ciò che si sta facendo, in modo da avere una base per la stesura dell'elaborato.

Infine, è richiesto agli studenti di consegnare la proposta di indice prima di iniziare la stesura, per poi consegnare ogni capitolo a mano a mano che questi vengono completati. La tesi completa va consegnata per la rilettura finale a relatore e correlatore almeno una settimana prima della consegna ufficiale, per permettere un ultimo *proof reading* e l'integrazione delle eventuali modifiche richieste.

Altri dettagli:

- E' estremamente importante che lo studente comunichi con i relatori per aggiornamenti regolari, in particolare ogni volta che ci sono sviluppi significativi o decisioni da prendere.
- E' altresì fondamentale che ogni comunicazione via mail avvenga mettendo sempre tutti i relatori/correlatori tra i destinatari.
- Di norma non serve venire a lavorare in laboratorio, tuttavia se lo studente lo preferisce è il benvenuto. Per ottenere il badge di accesso si acceda al portale: `https://badge.di.unimi.it/`

A.1 Aspetti Burocratici (CdL Triennali)

Solitamente l'attività di tirocinio interno per la triennale si articola nei seguenti passaggi formali:

1. Scegliere un argomento su cui lavorare: è possibile partire da una propria idea o scegliere un tema tra quelli proposti dai docenti
2. Scegliere relatore e correlatore: il relatore deve essere uno strutturato del Dipartimento (Ricercatore, Professore associato o Professore ordinario)
3. Definire insieme ai relatori: titolo (anche provvisorio), obbiettivi e tabella di marcia
4. Procedere con l'apertura formale del tirocinio sulla piattaforma `https://tirocini.di.unimi.it`
5. Il tirocinio deve durare almeno 14 settimane. Durante questo tempo si dovrebbe svolgere il grosso del lavoro
6. A tirocinio ultimato, se e solo se il relatore è d'accordo, si procede con la chiusura formale del tirocinio. Questa consiste da un lato nell'iscrizione all'appello apposito sul SIFA/UniMIA a carico dello studente, e dall'altro nell'inoltro della richiesta di chiusura a carico del relatore
7. Iscrizione alla sessione di laurea sul sito del Corso di Laurea `https://<nome_cdl>.cdl.unimi.it/it/studiare/laurearsi` (es. `https://informaticamusicale.cdl.unimi.it/it/studiare/laurearsi`)
8. Consegna di elaborato e riassunto in formato PDF-A (questo template produce già il formato richiesto) non superiore a 10MB
9. Discussione pubblica, che in quanto tale è aperta a spettatori esterni
10. Festeggiamenti (si ricorda che questi devono essere svolti nel rispetto dei luoghi e delle istituzioni rappresentate)

A.2 Aspetti Pratici

Relativamente gli aspetti pratici, ogni relatore ha un suo modus operandi per la gestione dei tesisti, quanto riportato di seguito va quindi considerato a solo titolo esemplificativo; in particolare questa è la procedura tipica seguita dagli studenti del Corso di Laurea di Informatica Musicale:

1. Analisi dello stato dell'arte (Cap. 2).
   E' importante annotare tutto sin da subito usando LaTeX, perché poi andrà scritto nell'elaborato.
2. Definizione del problema, "design" del sistema, scelta degli strumenti
3. Implementazione
4. Definizione ed esecuzione della campagna di test e/o validazione (Cap. 5)
5. Definizione dell'indice dell'elaborato
6. Scrittura dell'elaborato e correzione (meglio se in parallelo con gli altri passi)
7. Consegna dell'elaborato
8. Stesura della presentazione, correzione, suggerimenti

A.3 Gestione del Codice

Nella maggior parte dei tirocini dei CdL in Informatica, lo studente si troverà a scrivere del codice. Si consiglia di concordare in fase preliminare il modo di condivisione del codice. Per quanto riguarda il LIM, il codice pertinente alle tesi viene generalmente versionato su GitHub (`https://github.com/LIMUNIMI`).

Nel caso di utilizzo di git (con qualsiasi remoto, sia esso GitHub, GitLab, Bitbucket, ...), si consiglia allo studente di avere una infarinatura generale prima di utilizzarlo per il proprio progetto. Un buon tutorial interattivo è `https://learngitbranching.js.org`, anche se include argomenti avanzati che possono essere al di là delle necessità contingenti dello studente.

Appendice B

Documenti da Produrre

I membri del LIM in generale non richiedono agli studenti di consegnare una copia cartacea del proprio lavoro. In altri casi, si invita lo studente a concordare questo punto con il proprio relatore. In prossimità della discussione viene richiesto loro di condividere in formato digitale:

- la versione finale dell'elaborato;
- il riassunto caricato in piattaforma;
- il codice sviluppato;
- tutti i materiali di supporto utilizzati e/o prodotti;
- la presentazione predisposta per la discussione.

La consegna può avvenire via chiavetta USB o attraverso i sistemi di condivisione dei file (WeTransfer, Dropbox, Google Drive, ecc.).

B.1 Riassunto

In prossimità della data di laurea, viene richiesto agli studenti di caricare in piattaforma anche il riassunto. La sua funzione è permettere alla commissione di visionare rapidamente gli argomenti trattati da ciascun elaborato.

L'estensione di tale documento non dovrebbe superare le due pagine, e spesso una pagina è più che adeguata allo scopo. Riguardo i contenuti, il riassunto riprende – in maniera molto succinta – lo schema dell'elaborato finale, e quindi, ad esempio, la scaletta proposta in 1.4.

Visto lo scopo del documento, è buona prassi bilanciare il testo in modo da dedicare poche righe allo stato dell'arte, in favore delle parti originali dell'elaborato.

B.2 Presentazione

Gli studenti delle lauree triennali hanno a disposizione al massimo 10 minuti per presentare il proprio elaborato, che diventano 15 per le lauree magistrali.

Si raccomanda di provare ripetutamente il discorso cronometrandosi e considerando i tempi morti per l'accensione del PC, l'avvio dei software, il passaggio da un'applicazione a un'altra, e via dicendo. Si segnala inoltre la possibilità di prenotare l'aula della discussione per provare in anticipo i collegamenti e verificare che immagini e audio vengano correttamente riprodotti.

Per via degli stringenti vincoli temporali, difficilmente una presentazione multimediale può superare la soglia di 15 slide. Questo valore è puramente indicativo, perché dipende dalla densità di informazioni della slide e dal tempo dedicato a ciascuna slide mentre si sta presentando.

La scelta tecnologica più comune è Microsoft PowerPoint, ma qualsiasi alternativa (Prezi, Acrobat PDF, LaTeX, ecc.) è ammissibile.

Gli errori più comuni da evitare sono:

- slide dal contenuto testuale troppo ricco;
- slide che verranno lette parola per parola in fase di presentazione.

Entrambi i problemi si risolvono impostando il testo in maniera estremamente schematica, tendenzialmente per elenchi puntati. Si presti attenzione, però, a usare una forma omogenea. Ad esempio, sarebbe poco elegante il seguente elenco, che include aggettivi, sostantivi e frasi di senso compiuto, e non fa un uso corretto di iniziali maiuscole:

- applicativo esteticamente molto gradevole;
- C'è la possibilità di interagire con altri utenti;
- l'utente può esportare file in formato XML.

Una possibile revisione, basata solo sui sostantivi, è:

- qualità estetica;
- interazione tra utenti;
- supporto dell'XML.

Un altro consiglio per rendere la presentazione cognitivamente più gestibile da parte del pubblico è quello di aggiungere in calce ad ogni slide un indicatore dello stato di avanzamento (ad esempio "Slide 3 di 15"). In questo modo, in caso di esaurimento del tempo concesso la commissione saprà se dare modo di concludere ("Slide 14 di 15") o se

fermare il candidato ("Slide 14 di 138"). In assenza di un indicatore la commissione tende a presupporre il secondo caso.

Per quanto riguarda gli argomenti da trattare durante la discussione, è opportuno esporre brevemente lo stato dell'arte per concentrarsi sul proprio lavoro, sui test e sui risultati. Una tecnica per capire quanto tempo dedicare a ogni argomento consiste nel dare un peso a ciascuna parte di cui si vuole parlare, quindi assegnare un numero di minuti proporzionale a detto peso in modo da raggiungere il tempo massimo a disposizione.

Video dimostrativi e *live demo* sono gradite, ma è sconsigliabile dedicare troppo tempo a questi contributi, a meno che al contenuto multimediale si sovrapponga una spiegazione da parte del candidato.

Bibliografia

[1] Luca Surian. *L'autismo - Conoscerlo e affrontarlo.* Il Mulino, 2021.

?

Progetto sviluppato presso il Laboratorio di Ricerca
in collaborazione con l'Azienda Specifica
https://di.unimi.it/it/ricerca/risorse-e-luoghi-della-ricerca/laboratori-di-ricerca